LAPORAN PRAKTIKUM

STRUKTUR DATA

“STACK”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “STACK” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya Stack pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, krirtik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktium 2
BAB II PEMBAHASAN **3**
2.1 Langkah Kerja Praktikum 3
2.2 Analisis Hasil dan Pembahasan 12
BAB III PENUTUP **14**
3.1 Kesimpulan 14
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, terdapat berbagai macam struktur data yang digunakan untuk mengatur dan mengelola data agar lebih mudah diproses. Salah satu struktur data yang cukup sering digunakan adalah **stack**. Stack merupakan struktur data yang bekerja dengan prinsip **Last In First Out (LIFO)**, yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Konsep ini sering dijumpai dalam kehidupan sehari-hari, misalnya tumpukan buku atau piring.

Pada praktikum struktur data ini, stack dipelajari melalui beberapa contoh penerapan menggunakan bahasa pemrograman Java. Praktikum tidak hanya membahas penggunaan stack secara langsung dengan class `Stack`, tetapi juga bagaimana stack dapat diimplementasikan sendiri menggunakan array dan `ArrayList`. Selain itu, stack juga digunakan untuk menyelesaikan permasalahan tertentu seperti mencari nilai maksimum, menyimpan data objek mahasiswa, serta mengevaluasi ekspresi postfix.

Melalui praktikum ini, mahasiswa diharapkan tidak hanya memahami teori stack secara konsep, tetapi juga mampu melihat bagaimana prinsip LIFO diterapkan secara nyata di dalam program. Dengan mencoba berbagai studi kasus, mahasiswa dapat lebih memahami cara kerja stack serta kegunaannya dalam pemrograman.

1.2 Tujuan Praktikum

Tujuan praktikum:

1. Memahami konsep dasar struktur data stack dan prinsip kerja LIFO (Last In First Out).
2. Mempelajari penggunaan stack bawaan Java menggunakan class Stack.
3. Mengimplementasikan stack secara manual menggunakan array dan ArrayList.
4. Menggunakan operasi dasar stack seperti push, pop, peek, dan isEmpty dalam program.
5. Menerapkan stack untuk menyelesaikan permasalahan, seperti mencari nilai maksimum dalam stack.
6. Mengelola data objek (data mahasiswa) menggunakan struktur data stack.

7. Menerapkan stack dalam evaluasi ekspresi matematika postfix.
8. Melatih logika pemrograman dan ketelitian dalam mengelola data bertipe tumpukan.

1.3 Manfaat Praktium

Manfaat parktikum:

1. Mahasiswa menjadi lebih memahami cara kerja stack secara praktis, tidak hanya dari teori.
2. Mahasiswa mampu menggunakan stack untuk menyelesaikan berbagai permasalahan pemrograman.
3. Mahasiswa dapat membedakan penggunaan stack bawaan Java dan stack hasil implementasi sendiri.
4. Mahasiswa terlatih dalam menerapkan operasi dasar stack dengan benar agar tidak terjadi error seperti stack overflow atau stack underflow.
5. Mahasiswa memahami penerapan stack dalam kasus nyata, seperti pengolahan data dan evaluasi ekspresi matematika.
6. Praktikum ini membantu meningkatkan pemahaman mahasiswa terhadap struktur data sebagai dasar untuk materi lanjutan.

BAB II
PEMBAHASAN

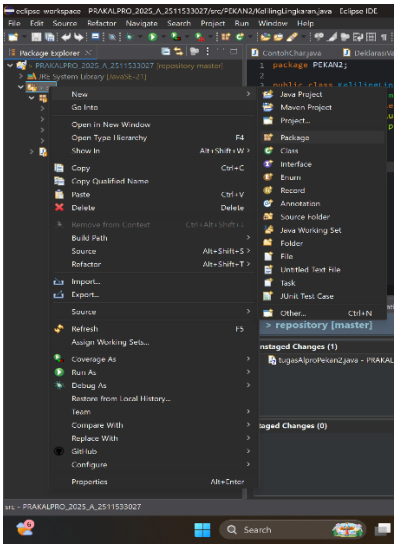
2.1 Langkah Kerja Praktikum

1. Persiapan Awal

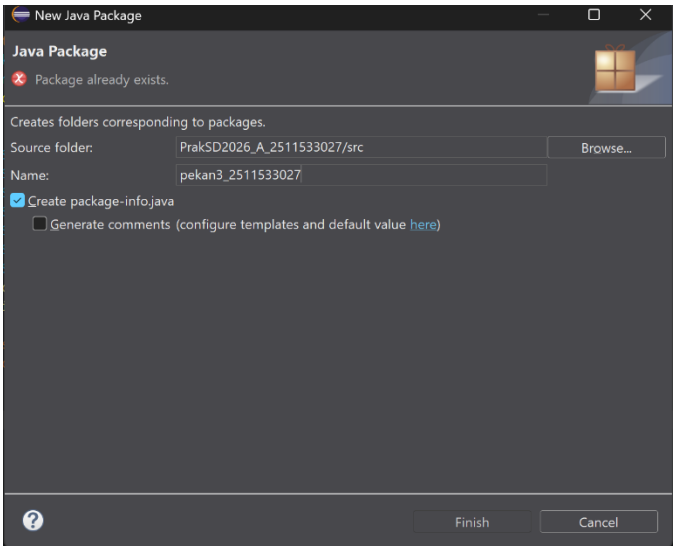
   a) Buka apl Eclipse IDE di laptop atau di komputer.

2. Membuat Package

   a) Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

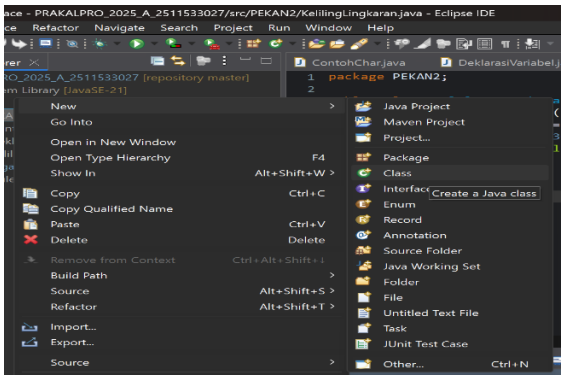


   b) Beri nama package **pekan3_NIM**.



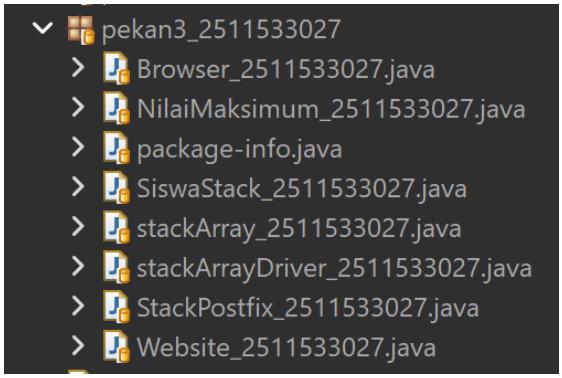
3. Membuat Class Baru untuk Setiap Percobaan

   a) Klik kanan pada package **pekan2_NIM**, lalu pilih **New**, terakhir klik **Class**.



   b) Buat beberapa kelas sesuai percobaan

      - NilaiMaksimum_NIM.java
      - SiswaStack_NIM.java
      - stackArray_NIM.java
      - stackArrayDriver_NIM.java
      - StackPostfix_NIM.java



4. Menulis Kode Program

   1. NilaiMaksimum_NIM.java

      - Program ini digunakan untuk memahami penerapan struktur data Stack dalam mencari nilai maksimum.
      - Stack bertipe data Integer digunakan untuk menyimpan beberapa nilai angka.
      - Beberapa elemen dimasukkan kedalam stack menggunakan method push().
      - Method peek() digunakan untuk melihat elemen teratas pada stack.
      - Program meggunakan stack tambahan sebagai backup agar isi stack utama tidak berubah.

- Proses pencarian nilai maksimum dilakukan dengan membandingkan setiap elemen stack.
- Method pop() dugunakan untuk mengambil elemen dari stack satu per satu.
- Setelah proses selesai, seluruh elemen stack dikembalikan ke stack semula.
- Hasil akhir berupa nilai maksimum dari elemen-elemen yang ada di dalam stack.

```java
package pekan3_2511533027;

import java.util.Stack;

public class NilaiMaksimum_2511533027 {
    public static int max_3027(Stack<Integer> s_3027) {
            Stack<Integer> backup_3027 = new Stack<Integer>();
            int maxValue_3027 = s_3027.pop();
            backup_3027.push(maxValue_3027);
            while (!s_3027.isEmpty()) {
                    int next_3027 = s_3027.pop();
                    backup_3027.push(next_3027);
                    maxValue_3027 = Math.max(maxValue_3027, next_3027);
            }
            while (!backup_3027.isEmpty()) {
                    s_3027.push(backup_3027.pop());
            }
            return maxValue_3027;
    }
    public static void main(String[] args) {
            Stack<Integer> s_3027 = new Stack<Integer>();
            s_3027.push(70);
            s_3027.push(12);
            s_3027.push(20);
             System.out.println("isi stack"+s_3027);
             System.out.println("Stack Teratas "+s_3027.peek());
             System.out.println("Nilai Maksimum "+max_3027(s_3027));
    }

}
```

2. SiswaStack_NIM.java

- Program ini digunakan untuk memahami penggunaan Stack dalam menyimpan data objek.
- Data yang disimpan berupa objek Siswa yang memiliki atribut nama dan NIM.
- Stack diimplementasikan menggunakan ArrayList sebagai media penyimpanan.
- Method push() digunakan untuk menambahkan data siswa ke dalam stack.
- Method pop() digunakan untuk menghapus data siswa yang berada di posisi paling atas.
- Method peek() digunakan untuk melihat data siswa teratas tanpa menghapusnya.
- Method isEmpty() digunakan untuk mengecek apakah stack masih kosong atau tidak.
- Seluruh isi stack ditampilkan dari elemen teratas ke terbawah untuk menunjukkan konsep LIFO.
- Program ini menunjukkan bahwa Stack dapat digunakan untuk menyimpan data selain tipe primitif.

```java
package pekan3_2511533027;

import java.util.ArrayList;

 class Siswa_2511533027 {
   String nama_3027;
   int nim_3027;

   public Siswa_2511533027(String nama_3027, int nim_3027) {
         this.nama_3027 = nama_3027;
         this.nim_3027 = nim_3027;
   }

   @Override
   public String toString() {
         return "Nim: " + nim_3027 + ", Nama: " + nama_3027;
   }
```

```
}
public class SiswaStack_2511533027 {
    private ArrayList<Siswa_2511533027> stack;

    public SiswaStack_2511533027() {
            stack = new ArrayList<>();
    }

    public void push_3027(Siswa_2511533027 mhs) {
            stack.add(mhs);
    }

    public Siswa_2511533027 pop_3027() {
            if (!isEmpty_3027()) {
                    return stack.remove(stack.size() - 1);
            }
            return null;
    }

    public Siswa_2511533027 peek_3027() {
            if (!isEmpty_3027()) {
                    return stack.get(stack.size() - 1);
            }
            return null;
    }

    public boolean isEmpty_3027() {
            return stack.isEmpty();
    }

    public void tampilkanSiswa_2511533027() {
            for (int i = stack.size() - 1; i >= 0; i--) {
                    System.out.println(stack.get(i));
            }
    }
    public static void main(String[] args) {
            SiswaStack_2511533027        studentStack        =        new
SiswaStack_2511533027();
```

```
        Siswa_2511533027 mhs1 = new Siswa_2511533027("Ali", 1);
        Siswa_2511533027 mhs2 = new Siswa_2511533027("Boby", 2);
        Siswa_2511533027 mhs3 = new Siswa_2511533027("Charles", 3);

        studentStack.push_3027(mhs1);
        studentStack.push_3027(mhs2);
        studentStack.push_3027(mhs3);

         System.out.println("Siswa di dalam stack:");
         studentStack.tampilkanSiswa_2511533027();

         System.out.println("siswa         teratas         "         +
studentStack.peek_3027());
         System.out.println("mengeluarkan siswa teratas dari stack: "
+ studentStack.pop_3027());
         System.out.println("daftar siswa setelah di pop :");
         studentStack.tampilkanSiswa_2511533027();
    }
}
```

3. stackArray_NIM.java
   - Program ini digunakan untuk memahami implementasi stack secara manual menggunakan array.
   - Stack dibuat dengan kapasitas maksimum sebanyak 1000 elemen.
   - Variabel top digunakan untuk menandai posisi elemen teratas dalam stack.
   - Method push() digunakan untuk menambahkan elemen ke dalam stack.
   - Program melakukan pengecekan kondisi stack overflow sebelum menambahkan data.
   - Method pop() digunakan untuk menghapus elemen teratas dari stack.
   - Program juga menangani kondisi stack underflow saat stack kosong.
   - Method peek() digunakan untuk melihat elemen teratas tanpa menghapusnya.
   - Method print() digunakan untuk menampilkan seluruh isi stack.

```
package pekan3_2511533027;

public class stackArray_2511533027 {
    static final int MAX = 1000;
```

```
int top_3027;
int a_3027[] = new int [MAX];
boolean isEmpty_3027()
{
        return (top_3027 < 0);
}
stackArray_2511533027()
{
        top_3027 = -1;
}
boolean push_3027(int x)
{
        if (top_3027 >= (MAX - 1)) {
                System.out.println("Stack Overflow");
                return false;
        }
        else {
               a_3027[++top_3027] = x;
                System.out.println(x + "dimasukkan dalam stack");
                return true;
        }
}
int pop_3027()
{
        if (top_3027 < 0) {
                System.out.println("Stack Underflow");
                return 0;
        }
        else {
               int x = a_3027[top_3027--];
               return x;
}

}
int peek_3027()
{
        if (top_3027 < 0) {
                System.out.println("Stack Underflow");
                return 0;
        }
```

```
        else {
                int x = a_3027[top_3027];
                return x;
        }
    }
    void print() {
        for(int i = top_3027;i>-1;i--) {
                System.out.print(" "+ a_3027[i]);
        }
    }
}
```

4. stackArrayDriver_NIM.java
   - Program ini berfungsi sebagai driver untuk menjalankan class stackArray.
   - Objek stack dibuat untuk menguji method yang telah didefinisikan sebelumnya.
   - Beberapa elemen dimasukkan ke dalam stack menggunakan method push()
   - Method pop() digunakan untuk mengeluarkan elemen teratas dari stack.
   - Method peek() digunakan untuk melihat elemen yang berada di posisi paling atas.
   - Program menampilkan isi stack setelah proses push dan pop dilakukan.
   - Class ini membantu memastikan bahwa implementasi stack array berjalan dengan benar.

```
package pekan3_2511533027;

public class stackArrayDriver_2511533027 {
    public static void main(String[] args) {
        stackArray_2511533027 s = new stackArray_2511533027();
        s.push_3027(10);
        s.push_3027(20);
        s.push_3027(30);
         System.out.println(s.pop_3027() + " dikeluarkan dari stack");
         System.out.println("Elemen     teratas     adalah     :"     +
s.peek_3027());
         System.out.println("Elemen pada stack :");
         s.print();
    }
```

```
}
```

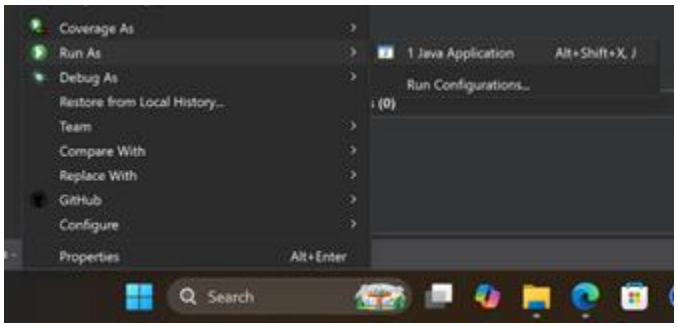
5. StackPostfix_NIM.java
    - Program ini digunakan untuk memahami penerapan stack dalam evaluasi ekspresi postfix.
    - Ekspresi postfix dimasukkan dalam bentuk string.
    - Stack bertipe Integer digunakan untuk menyimpan operand.
    - Scanner digunakan untuk membaca setiap token dalam ekspresi postfix.
    - Jika token berupa angka, maka angka tersebut dimasukkan ke dalam stack menggunakan push().
    - Jika token berupa operator, maka dua operand teratas diambil menggunakan pop().
    - Operasi aritmatika dilakukan sesuai operator yang dibaca.
    - Hasil operator dimasukkan Kembali ke dalam stack.
    - Hasil akhir evaluasi ekspresi postfix diambil dari elemen terakhir stack.

```java
package pekan3_2511533027;

import java.util.Scanner;

public class StackPostfix_2511533027 {
    public static int postfixEvaluate_2511533027(String  expression) {
        Stack<Integer> s_3027 = new Stack<Integer>();
        Scanner input_3027 = new Scanner(expression);
        while (input_3027.hasNext()) {
            if(input_3027.hasNextInt())  { // an operand integer
                s_3027.push(input_3027.nextInt());
            } else {
                String operator_3027 = input_3027.next();
                int operand2 = s_3027.pop();
                int operand1 = s_3027.pop();
                if (operator_3027.equals("+")) {
                    s_3027.push(operand1 + operand2);
                } else if (operator_3027.equals("-")) {
                    s_3027.push(operand1 - operand2);
                } else if (operator_3027.equals("*")) {
                    s_3027.push(operand1 * operand2);
                } else {
                    s_3027.push(operand1 / operand2);
                }
            }
        }
        input_3027.close();
        return s_3027.pop();
    }
    public static void main(String[] args) {
         System.out.println("hasil postfix= "+postfixEvaluate_2511533027("5 2 4 * + 7 -"));
    }
}
```

5. Kompilasi dan Menjalankan Program



2.2 Analisis Hasil dan Pembahasan

1. NilaiMaksimum_NIM.java
   a) Hasil: Program berhasil mencari nilai maksimum dari beberapa elemen yang tersimpan di dalam stack. Elemen-elemen stack dapat ditampilkan terlebih dahulu, kemudian program dapat menentukan nilai terbesar tanpa mengubah urutan isi stack. Hasil nilai maksimum ditampilkan dengan benar setelah proses pencarian selesai.
   b) Analisis: Pada program ini, struktur data Stack digunakan untuk menyimpan data bertipe Integer. Proses pencarian nilai maksimum dilakukan dengan cara mengambil elemen stack satu per satu menggunakan method pop(), kemudian membandingkannya untuk mendapatkan nilai terbesar. Agar isi stack tidak hilang, digunakan stack tambahan sebagai penampung sementara. Setelah proses selesai, seluruh elemen dikembalikan ke stack awal. Program ini menunjukkan penerapan stack dalam pengolahan data numerik serta pemahaman konsep LIFO secara tidak langsung.
2. SiswaStack_NIM.java
   a) Hasil: Program berhasil menyimpan dan menampilkan data siswa menggunakan stack. Data siswa dapat ditambahkan ke dalam stack, ditampilkan dari urutan teratas, serta dapat dihapus menggunakan operasi pop. Selain itu, program juga dapat menampilkan siswa yang berada di posisi teratas tanpa menghapusnya.
   b) Analisis: Pada program ini, stack digunakan untuk menyimpan objek bertipe Siswa yang memiliki atribut nama dan NIM. Stack diimplementasikan menggunakan ArrayList sebagai media penyimpanan data. Operasi push(), pop(), dan peek() digunakan untuk mengelola data sesuai prinsip LIFO. Program ini menunjukkan bahwa stack tidak hanya dapat digunakan untuk tipe data sederhana, tetapi juga dapat digunakan untuk menyimpan data objek secara terstruktur.
3. stackArray_NIM.java
   a) Hasil: Program berhasil mengimplementasikan stack menggunakan array. Elemen dapat ditambahkan ke dalam stack hingga batas maksimum, dikeluarkan dari stack, serta ditampilkan dengan urutan dari elemen teratas. Program juga mampu menampilkan pesan ketika terjadi kondisi stack penuh atau stack kosong.

b) Analisis: Pada program ini, stack diimplementasikan secara manual menggunakan array dengan ukuran tetap. Variabel top digunakan untuk menandai posisi elemen teratas. Method push() dan pop() digunakan untuk mengelola data, serta dilakukan pengecekan kondisi overflow dan underflow. Program ini membantu memahami cara kerja stack di tingkat yang lebih dasar, tanpa menggunakan library bawaan Java.

4. stackArrayDriver_NIM.java
   a) Hasil: Program berhasil menjalankan dan menguji class stackArray. Beberapa elemen dapat dimasukkan ke dalam stack, kemudian satu elemen teratas berhasil dikeluarkan dan ditampilkan. Program juga dapat menampilkan elemen teratas serta seluruh isi stack setelah operasi dilakukan.
   b) Analisis: Class driver digunakan untuk menguji fungsi-fungsi yang terdapat pada class stackArray. Dengan adanya driver, proses pengujian method seperti push(), pop(), dan peek() menjadi lebih jelas. Program ini membantu memastikan bahwa implementasi stack menggunakan array telah berjalan sesuai dengan yang diharapkan.
5. StackPostfix_NIM.java
   a) Hasil: Program berhasil mengevaluasi ekspresi postfix dan menghasilkan nilai akhir yang sesuai dengan perhitungan. Operasi aritmatika seperti penjumlahan, pengurangan, perkalian, dan pembagian dapat dijalankan dengan benar menggunakan stack.
   b) Analisis: Pada program ini, stack digunakan untuk menyelesaikan ekspresi postfix dengan cara menyimpan operand dan memproses operator sesuai urutan. Ketika operator ditemukan, dua operand teratas diambil dari stack, kemudian dilakukan operasi aritmatika. Hasil perhitungan dimasukkan kembali ke dalam stack. Program ini menunjukkan salah satu penerapan stack yang sering digunakan dalam pemrosesan ekspresi matematika.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Stack merupakan salah satu struktur data yang bekerja dengan prinsip Last In First Out (LIFO), di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Konsep ini dapat diterapkan dalam berbagai kasus pemrograman, baik untuk pengolahan data sederhana maupun permasalahan yang lebih kompleks.

Melalui beberapa program yang dibuat, mahasiswa dapat memahami penggunaan stack menggunakan class Stack bawaan Java maupun implementasi stack secara manual menggunakan array dan ArrayList. Praktikum ini juga menunjukkan bahwa stack dapat digunakan untuk menyimpan berbagai jenis data, mulai dari tipe data primitif hingga objek, serta dapat dimanfaatkan untuk menyelesaikan permasalahan seperti pencarian nilai maksimum dan evaluasi ekspresi postfix.

Dengan melakukan praktikum ini, mahasiswa tidak hanya memahami konsep stack secara teori, tetapi juga mampu mengimplementasikan dan menguji langsung cara kerja stack dalam program Java. Praktikum ini menjadi dasar yang penting untuk memahami struktur data lain yang lebih lanjut serta penerapannya dalam dunia pemrograman.

DAFTAR PUSTAKA

[1] R. S. Wahyudi, "Struktur Data: Stack (LIFO)," Modul Praktikum Struktur Data, Program Studi Informatika, Universitas Andalas, 2026.

[2] Oracle Corporation, "Class Stack," Oracle Java Documentation. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html.

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“STACK”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Stack menggunakan studi kasus Website dan Browser. Pada praktikum ini, penulis membuat program Java yang dapat mengelola data Website seperti menambah, menampilkan, dan menghapus data.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
1.4 Instruksi Tugas 1
BAB II PEMBAHASAN **3**
2.1 Kelas Website_NIM 3
2.2 Kelas Browser_NIM 3
BAB III PENUTUP **5**
3.1 Kesimpulan 5
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Stack merupakan salah satu struktur data yang bekerja dengan prinsip **Last In First Out (LIFO)**, yaitu data yang terakhir masuk akan menjadi data pertama yang keluar. Pada tugas ini, konsep stack diterapkan dalam simulasi **browser history**, di mana setiap website yang dikunjungi akan disimpan ke dalam stack dan website terakhir akan dihapus saat tombol *back* digunakan. Melalui program ini, konsep stack dapat dipahami secara lebih nyata melalui contoh kasus sederhana.

1.2 Tujuan

Tujuan:

1. Memahami konsep dasar struktur data stack dan prinsip LIFO.
2. Menerapkan operasi stack seperti push, pop, dan peek pada Java.
3. Menggunakan stack untuk menyimpan data website berupa judul dan URL.
4. Membuat simulasi browser history sederhana menggunakan stack.

1.3 Manfaat

Manfaat:

1. Membantu memahami cara kerja stack secara praktis.
2. Melatih penggunaan struktur data stack dalam pemrograman Java.
3. Mempermudah pemahaman konsep LIFO melalui simulasi browser history.
4. Menjadi dasar untuk penerapan stack pada kasus lain yang lebih kompleks.

1.4 Instruksi Tugas

1. Aturan Penamaan
   a) Nama Kelas: Gunakan akhiran NIM lengkap.
      Contoh : Website_2311532008 dan Browser_2311532008.
   b) Nama Variabel Atribut: Nama Variabel Atribut : Setiap variabel atribut di kelas ADT gunakan akhiran 4 digit terakhir NIM.
      Contoh (NIM ...2008): String judul_2008, String url_2008.
2. Struktur Program
   1. Kelas ADT Website (Website_NIMLengkap)
      - Atribut:

    - Judul Website
    - URL Website
  - Method:
    - Constructor untuk menginisialisasi atribut.
    - Selektor (Getter) untuk setiap atribut.
    - Mutator (Setter) untuk setiap atribut.
2. Kelas Driver (Browser_NIMLengkap)
  - Kunjungi Website (Push) : Memasukkan data website baru yang sedang dikunjungi ke dalam tumpukan.
  - Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas) dan menampilkan judul halaman yang berhasil dihapus.
  - Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang aktif dibuka tanpa menghapusnya dari tumpukan.
  - Cek Status History : Menampilkan jumlah total riwayat yang tersimpan atau mengecek apakah riwayat kosong menggunakan .isEmpty().
3. Contoh Output Program

   === Browser History NIM: 2411532007 ===

   1. Kunjungi Website (Push)
   2. Tombol Back (Pop)
   3. Lihat Halaman Aktif (Peek)
   4. Keluar

   Pilihan: 1

   Masukkan Judul: SSO UNAND

   Masukkan URL: https://sso.unand.ac.id/login

   Berhasil mengunjungi halaman!

BAB II
PEMBAHASAN

2.1 Kelas Website_NIM

Kelas **Website_2511533027** digunakan untuk merepresentasikan data sebuah website yang akan disimpan ke dalam browser history. Di dalam kelas ini terdapat atribut judul dan URL yang menyimpan informasi website. Kelas ini dilengkapi dengan constructor, getter, dan setter agar data website bisa diakses dan diubah dengan mudah. Selain itu, method toString() digunakan untuk menampilkan informasi website dalam bentuk teks. Kelas ini berperan sebagai objek yang nantinya akan dimasukkan ke dalam stack.

```java
package pekan3_2511533027;

public class Website_2511533027 {
    private String judul_3027;
    private String url_3027;

    // Constructor
    public Website_2511533027(String judul_3027, String url_3027) {
        this.judul_3027 = judul_3027;
        this.url_3027 = url_3027;
    }

    // Getter
    public String getJudul_3027() {
        return judul_3027;
    }
    public String getUrl_3027() {
        return url_3027;
    }

    // Setter
    public void setJudul_3027(String judul_3027) {
        this.judul_3027 = judul_3027;
    }
    public void setUrl_3027(String url_3027) {
        this.url_3027 = url_3027;
    }

    @Override
    public String toString() {
        return "Judul: " + judul_3027 + ", URRL: " + url_3027;
    }

}
```

2.2 Kelas Browser_NIM

Kelas **Browser_2511533027** merupakan bagian utama yang menjalankan program simulasi browser history. Pada kelas ini digunakan **Stack** untuk menyimpan objek Website sesuai dengan konsep LIFO. Program menyediakan menu yang memungkinkan user untuk mengunjungi website, kembali ke halaman sebelumnya, melihat halaman yang sedang aktif, dan mengecek status history. Input dari user dibaca menggunakan **Scanner**, sehingga program dapat berjalan secara interaktif. Menu dijalankan menggunakan perulangan **do-while** agar terus muncul sampai user memilih keluar. Dengan program ini, simulasi browser history sederhana dapat dijalankan secara dinamis.

```java
package pekan3_2511533027;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511533027 {
    public static void main(String[] args) {
        Stack<Website_2511533027> history_3027 = new Stack<>();
        Scanner input_3027 = new Scanner(System.in);
        int choice_3027;

        do {
            System.out.println("\n=== Browser History NIM: 2511533027 ===");
            System.out.println("1. Kunjungi Website (Push).");
            System.out.println("2. Tombol Back (Pop).");
            System.out.println("3. Lihat Halaman Aktif (Peek).");
            System.out.println("4. Cek Status History.");
            System.out.println("5. Keluar.");
            System.out.print("Pilihan : ");
            choice_3027 = input_3027.nextInt();
            input_3027.nextLine();

            switch (choice_3027) {
            case 1:
                System.out.print("Masukkan Judul Website: ");
                String judul_3027 = input_3027.nextLine();
                System.out.print("Masukkan URL Website: ");
                String url_3027 = input_3027.nextLine();

                Website_2511533027 web = new Website_2511533027(judul_3027, url_3027);
                history_3027.push(web);

                System.out.println("Berhasil Mengunjungi Halaman!");
                break;
```

```java
            case 2:
                if (history_3027.isEmpty()) {
                    System.out.println("History Kosong, tidak bisa kembali.");
                } else {
                    Website_2511533027 hapus = history_3027.pop();
                    System.out.println("Kembali dari halaman: " + hapus.getJudul_3027());
                }
                break;

            case 3:
                if (history_3027.isEmpty()) {
                    System.out.println("Tidak ada halaman yang sedang dibuka.");
                } else {
                    System.out.println("Halaman aktif:");
                    System.out.println(history_3027.peek());
                }
                break;

            case 4:
                if (history_3027.isEmpty()) {
                    System.out.println("History masih kosong.");
                } else {
                    System.out.println("Jumlah history tersimpan: " + history_3027.size());
                }
                break;

            case 5:
                System.out.println("Terimakasih, browser ditutup.");
                break;

            default:
                System.out.println("Pilihan tidak valid.");
            }

        } while (choice_3027 !=5);

        input_3027.close();
    }

}
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan tugas yang telah dibuat, dapat disimpulkan bahwa struktur data **Stack** bekerja dengan prinsip **Last In First Out (LIFO)** dan dapat diterapkan dalam simulasi browser history. Program yang dibuat mampu menyimpan data website, menampilkan halaman aktif, serta menghapus halaman terakhir yang dikunjungi menggunakan operasi stack. Melalui tugas ini, pemahaman mengenai penggunaan stack dalam pemrograman Java menjadi lebih jelas dan aplikatif, khususnya dalam pengelolaan data yang bersifat dinamis.

DAFTAR PUSTAKA

[1] R. S. Wahyudi, "Struktur Data: Stack (LIFO)," Modul Praktikum Struktur Data, Program Studi Informatika, Universitas Andalas, 2026.

[2] Oracle Corporation, "Class Stack," Oracle Java Documentation. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html.